biohub-cell-tracking-during-development

1位 Solution

Rank	1
Team	Sergio Alvarez
Author	Sergio Alvarez
Topic ID	744801
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/biohub-cell-tracking-during-development/discussion/744801
Retrieved with Kaggle CLI on 2026-10-04.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

このような素晴らしいコンペを開催してくださった BioHub と Kaggle に感謝します。2年前に Kaggle のコンペに参加し始めたときには、ここまで来られるとは想像もできませんでした。初のソロ金メダル、そして1位を取れたことをとても誇りに思います。

この解法を読む際には、これがゼロから設計されたものではないことを念頭に置いてください。アイデアが浮かぶたびに追加されながら成長してきたもので、そのアイデアも一直線に出てきたわけではなく、多くのことを試しました。そのため、スコアを落とさずに（もしかすると改善しながら）もっと単純で洗練された形にできる部分もあるかもしれません。

私はコンペ開始からわずか4時間後に参加しました。トラッキング全般についてはあまり知らず、ましてや細胞トラッキングについてはなおさらでした。しかしソロ金を狙いたかったので、このテーマを学ぶことに非常に力を注ぎました。

コンペの序盤、タスクに慣れようとしていたとき、コンピュータビジョンの他の分野とは対照的に、細胞トラッキングが「とにかくスケールアップしよう」という潮流に乗っていないことにとても興味を惹かれました。実際、主に使われているのは形状、輝度、速度などの「手作り」の特徴量で、これらの特徴量は盲目的です。つまり、細胞そのものを一度も見ていないのです。これはコンペを通じてずっと頭に残っていたことの一つであり、私のアイデアが巡り続けたテーマの一つでもあります。

2つ目の主な原動力は（スコアの改善にはそれほどではありませんでしたが）アイデア面で助けになったもので、YouTube で視聴した ultrack のプレゼンテーションでの一言です。ホストの一人である Jordão Bragantini [1] が「トラッキングが難しいのは、セグメンテーションが難しいからだ。もしセグメンテーションが解決されていれば、トラッキングはずっと簡単になるだろう…」と述べていました。このため、コンペの前半では主に検出部分の改善に取り組みましたが、ほとんど進展はありませんでした。セグメンテーション部分に取り組んだこの1か月半は、最終的に細胞を手作業でアノテーションするという苦肉の策に私を導きました。学部時代の研究や修士課程でこの種の手作業には慣れていましたが、いやはや、3D の細胞をアノテーションするのは本当に大変です。多くの時間を費やしたにもかかわらず、アノテーションできたのは約1,800個の細胞だけで、既に提供されていた量の2%にも満たず、検出部分にとってはほとんど何も得られない割に労力がかかりすぎました。

それでも、やったことを後悔はしていません。何十時間も細胞を眺め続けたことで、支配的な手作り特徴量から脱却するための多くのアイデアが得られたからです。そう、トラッキングは難しいです。細胞同士は極めて似ていて、絶えず変化し、さらに悪いことに分裂（DIVIDE）します。しかし、それまで一度も細胞をトラッキングしたことのない人間である私でも、3フレームを見て「これはフレーム t-1 の細胞 X で、これは t の細胞 X、そしてこれが t+1 でのその娘細胞だ」と自信を持って言えます。であれば、機械も何らかの形でこれを学習できるはずです。

これが Biohub - Cell Tracking During Development の1位解法です。

アプローチの概要

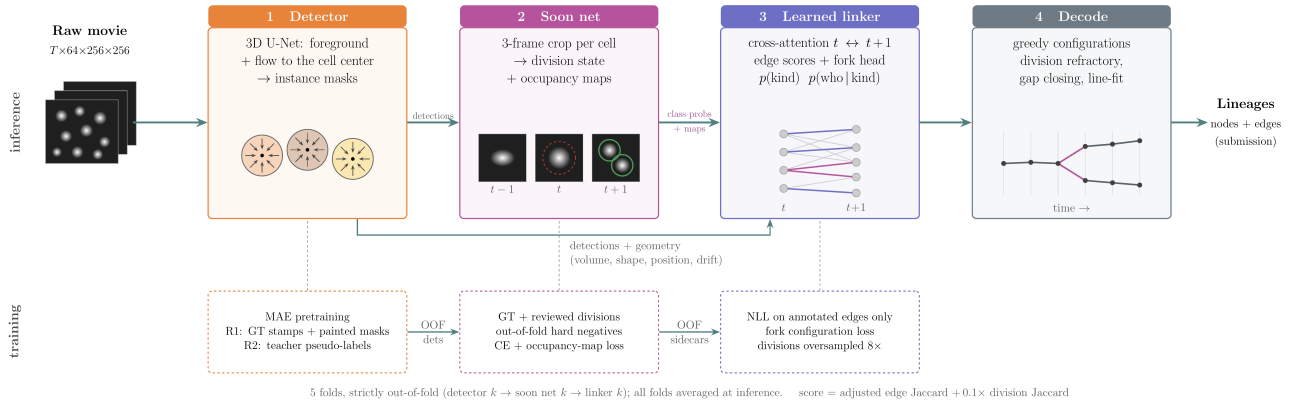
私の解法は tracking-by-detection のパラダイムに従っており、3つの主要なステージとデコードで構成されています。

第1ステージである検出部分は、Cellpose スタイルのヘッド [2] を持つ素の U-Net で、foreground 確率と各細胞の中心を指すフローフィールドを予測します。このモデルは MAE [3] で事前学習し、手作業のアノテーション + GT ノード上の 4.305 µm のスタンプでファインチューニングし、その後、この最初のモデルで生成した pseudo label で再学習しました。

第2ステージはこのアプローチの面白い部分で、ここでは Soon Net と呼ぶことにします。各細胞の周りを切り出した3フレームを見て、その細胞の分裂状態（interphase、もうすぐ分裂する（soon to divide）、分裂直後（just divided））、どこにいたか、そしてどこへ行ったかを予測しようとします。アーキテクチャは非常に小さな CNN エンコーダ（236K パラメータ）+ transformer（508K パラメータ）です。

第3ステージは学習型の linker で、これも transformer であり、LightGlue [4] からいくつかの着想を得ています。検出器の形状情報と Soon Net の出力を使って、フレーム t と t+1 の間の edge をスコアリングし、division を判定します。

最後に、greedy デコードによって linker のスコアを系譜（lineage）に変換します。概要は下の画像でも確認できます。



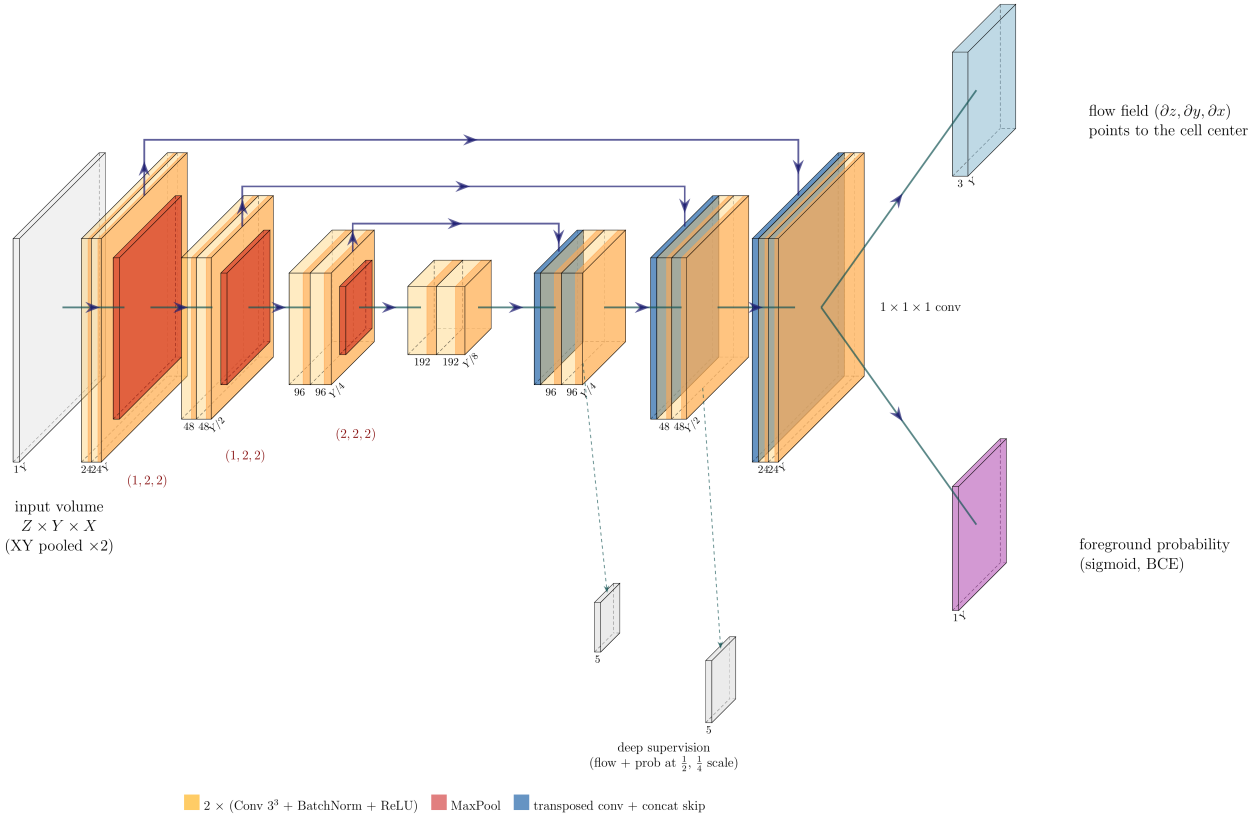
検出

検出器は Cellpose スタイルのヘッドを持つ素の 3D U-Net です。パラメータ数は約 3.1M で、サイズ 128×128×64 のクロップで学習しました。エンコーダは3つのダウンサンプリングステージ（24、48、96 チャネル）とそれに続く 192 チャネルのボトルネックからなり、各ブロックは 2 × (3×3×3 conv + BatchNorm + ReLU) です。プーリングは異方性で、(1,2,2)、(1,2,2)、(2,2,2) です。デコーダの最後では、2つの 1×1×1 ヘッドが foreground 確率と、細胞中心を指す3チャネルのフローフィールドを予測します。また、½ および ¼ 解像度での deep supervision も使っています。

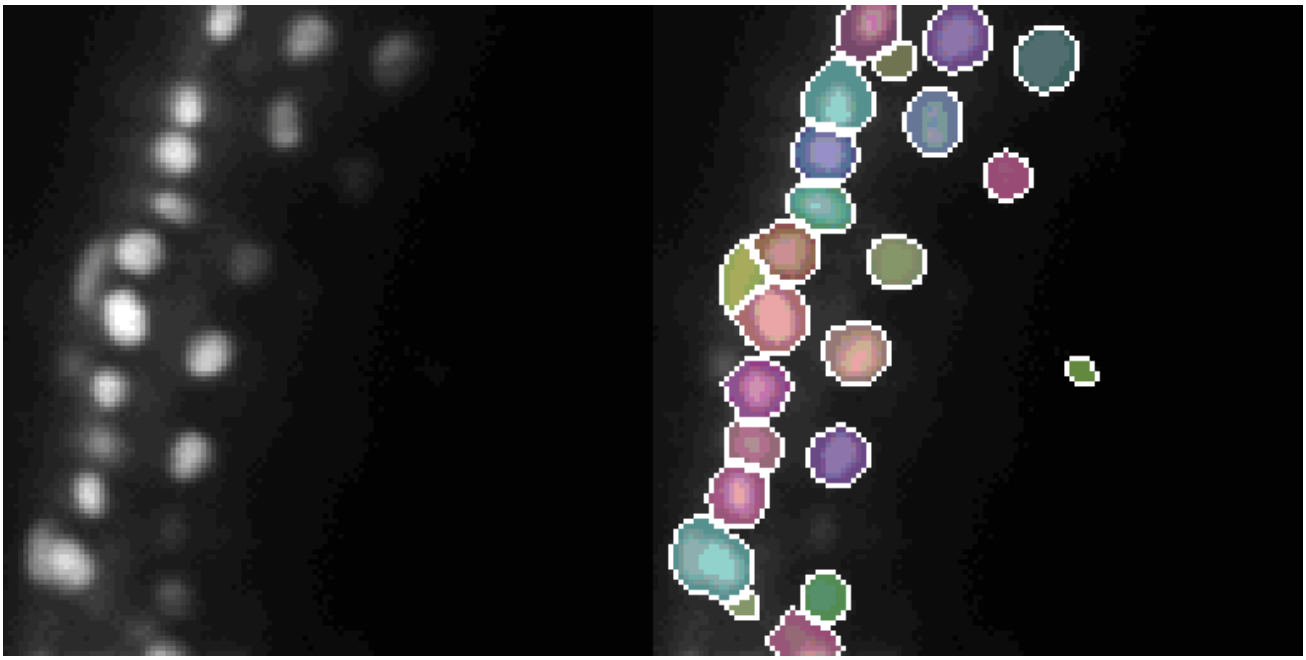
インスタンスを得るには、Cellpose と同様に、確率 > 0.5 のすべてのボクセルがフローをたどって細胞中心で止まるまで移動し、同じ場所（3 µm 以内）に行き着いたボクセルを1つのインスタンスとします。10 ボクセルより小さいインスタンスは除外します。

異方性プーリングは、私があまり確信を持てていないパラメータの一つです。データの Z 方向の異方性に対処する方法として試したところ、loss カーブがわずかに低くなったので残しました。いずれにせよ、おそらく大した効果はないでしょう…

アーキテクチャは下の図のとおりです。



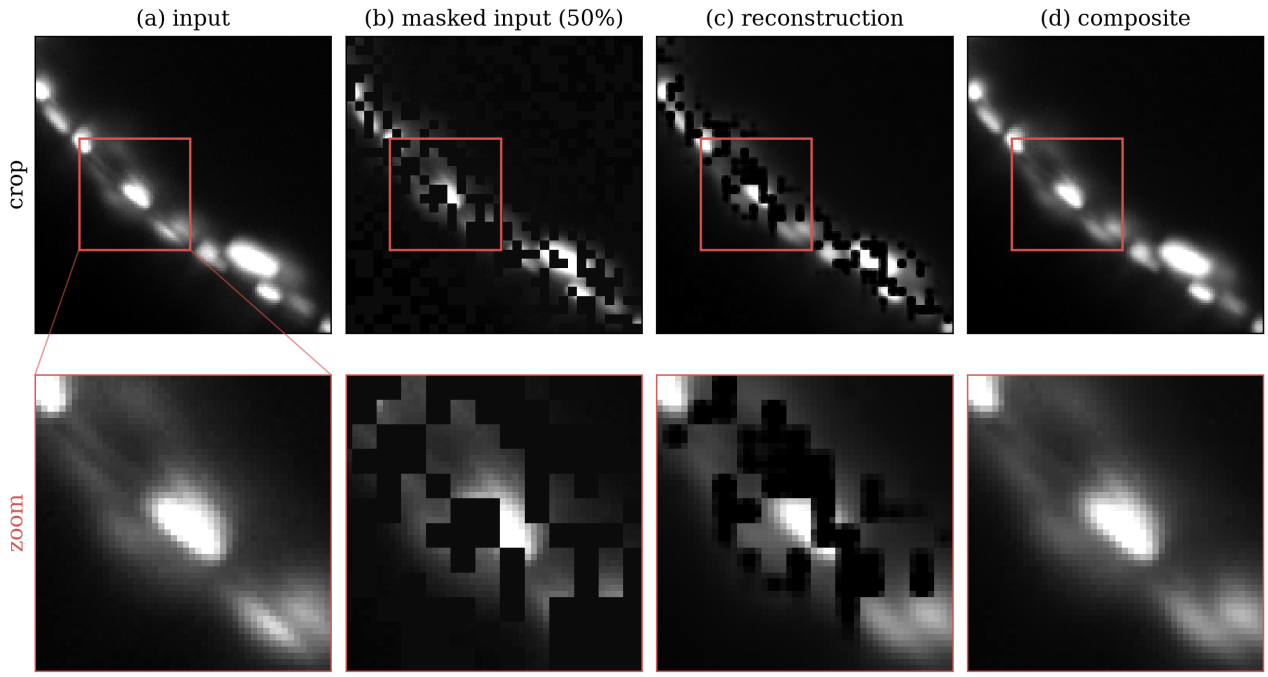
CZII や BYU といった他のコンペで主流だった Gaussian ブロブではなく、マスク出力を試すことにしたのは、HOCT [5] のような多くのトラッキング論文が形状特徴量を使っており、Ultrack [6] もマスクを必要とするためで、単純な Gaussian ブロブではそれらを提供できないからです。



モデルはまず、非常に「軽い」MAE（Masked Autoencoder）で事前学習しました。通常、MAE はフレームの大部分を大きなブロックでマスクします。私はずっと軽いバージョンを選び、体積の 50% を 2×4×4 ボクセル（各辺約 3.25 µm、核のおよそ3分の1）の小さなブロックでマスクしました。細胞は非常に小さいので、大きなブロックを使うのは理にかなわないと考えました。モデルが細胞を完全に幻覚（hallucinate）しなければならなくなるからです。軽い MAE であれば、幻覚することなく有用な特徴を学習できます。コンペの 180 本の動画（18,000 フレー

ム、再構成の確認用にさらに 15 本をホールドアウト）と、ultrack のゼブラフィッシュ胚の外部データ 1,500 フレーム（150 クロップ × 10 時点）を使い、30 エポック学習しました。正直なところ、これがメトリクスをどれだけ改善したのか、そもそも改善したのかどうかさえ断言できません。初期の実験では胚単位の外部分割（out-of-embryo split）で取り組んでおり、recall にわずかな改善が見られたので使い続けました。Private LB で遭遇し得る胚の多様性に対して、検出器をより頑健にしておきたかっただけです。そして最後に、たとえスコアが改善しなくても、モデルの収束がずっと速くなったので、悪いことであるはずがありません。この判断は文献にも裏付けられており、@hengck23 がディスカッションで共有した FOCUS-3D 論文 [7] でも MAE 事前学習が使われています。

MAE のコードは素晴らしい MIC-DKFZ グループによるもので、彼らの github [8] で入手できます。下の図は MAE の再構成結果で、実際に再構成を学習できていることがわかります。



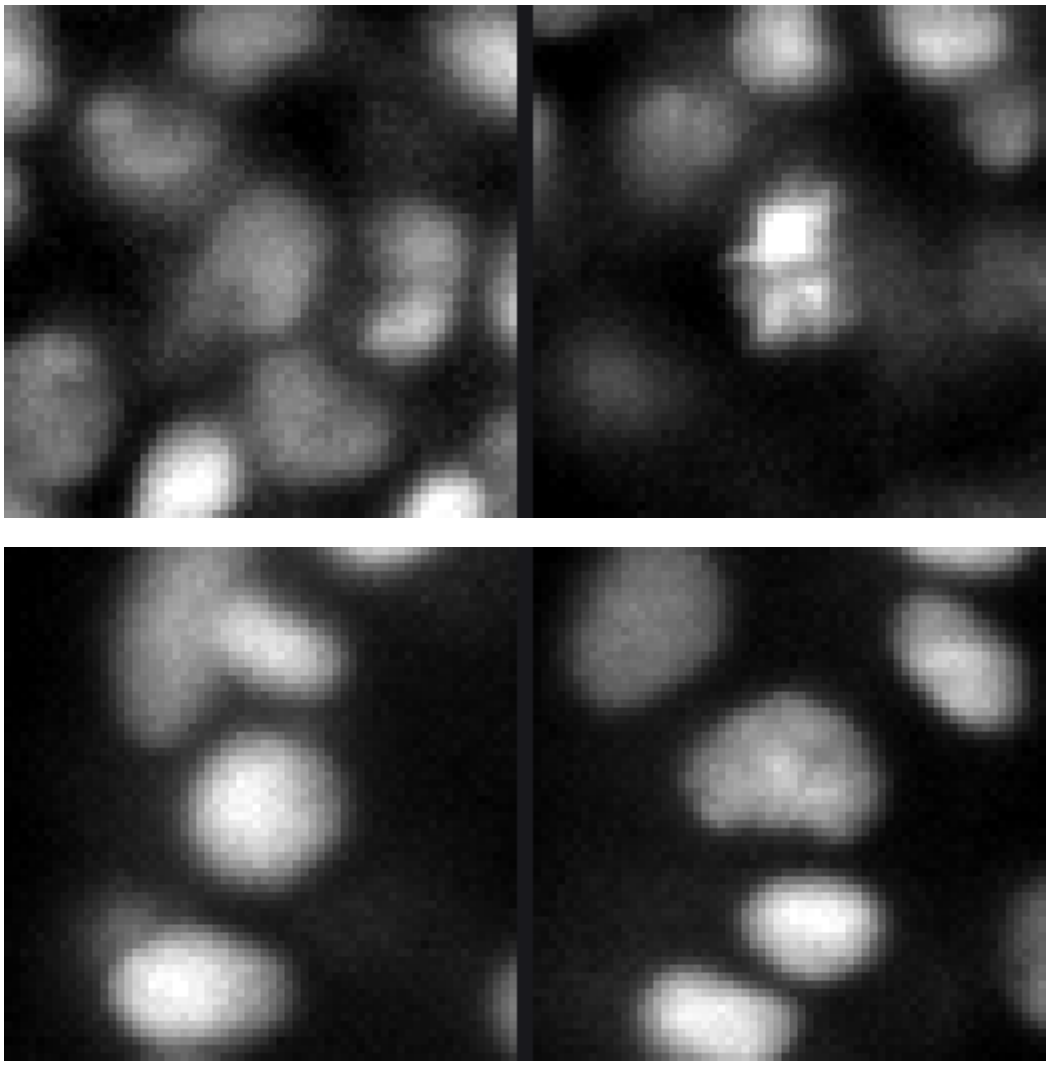
最初のファインチューニングでは、私が手で塗ったマスク（583 フレームにわたる約 1,800 細胞）と、アノテーション済みの 133k 個の GT 細胞上の 4.305 µm のスタンプを使いました。モデルは強いデータ拡張と EMA を使って 40 エポック学習しました。

最初のファインチューニングの後、5-fold 平均 + 4x flip TTA を使って、全 195 本の動画 + 150 個の外部クロップに対して pseudo label を生成しました。その後、高信頼度の検出のみを残し、公開されている twoPass トラッカーでトラックを生成し、高信頼度の検出のみを残すのに役立つよう min length > 3 でフィルタリングしました。はい、これは CV へのリークになるので Round 2 の検証スコアは楽観的ですが、GT がスパースであり、改善を目視で確認できたため、このトレードオフを受け入れました。

最終サブミッションでは 5 folds × 4x TTA を使い、1フレームあたり 20 回の forward pass となりました。

Soon to Divide / 視覚的トラッキング

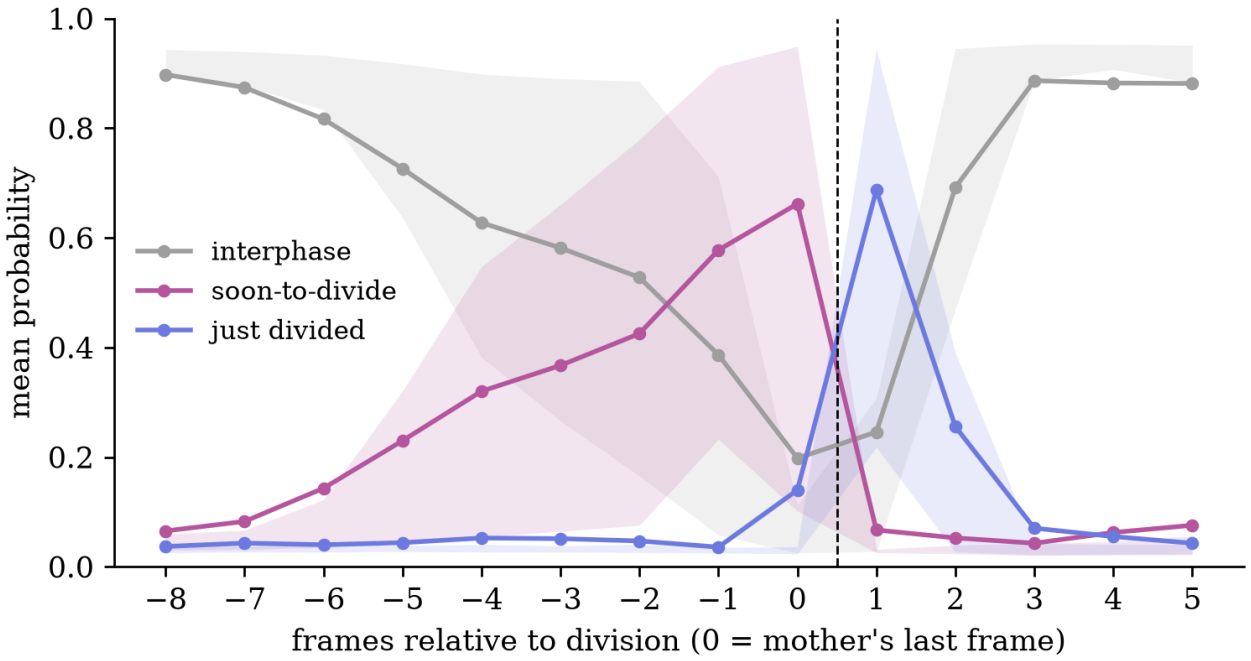
検出器のために手作業でアノテーションをしていたとき、division は目で見ると非常に見つけやすいことに気づきました。細胞は、少なくともゼブラフィッシュ胚では、かなり定型的なプロセスをたどります。揺れ始め、明るくなり、そして大きくなり、突然2つに弾けるのです。最初のアイデアはこれを見たことから生まれました。分裂が起こるところを見なくても、その細胞がもうすぐ分裂することがわかったのです。信じられませんか？下の gif でご自身で確かめてみてください。



分裂そのものは見せていませんが、どの細胞が分裂しようとしているか（右側の細胞）を目で見分けられたのではないでしょうか。

このコンセプトから最初のモデルが生まれました。当初の目的は、linker 用に Soon-to-Divide、Just-Divided、Interphase の3つの特徴量を作ることだけで、これら3クラスの確率は好きなように使えるというものでした。soon-to-divide クラスには、細胞に何かが起きていると目で見てわかる有糸分裂の段階、おそらく前期・中期・後期が含まれますが、学術的な名称や、有糸分裂のすべての段階を確実に識別することは気にしていませんでした。細胞に何かが起きていれば、ラベルを付けました。

私が思い描いていたのは、soon-to-divide の確率が t+1 で娘細胞が現れるピークまで直線的に増加し、その後 just divided に急に切り替わり、それによって linker が division の発生を知る、というものでした。最終的には soon-to-divide を単純化し、ラベル付けしたすべてのフレームではなく、母細胞の最後の2フレーム t-1 と t のみとし、娘細胞は t+1 に現れるものとしました。下のプロットに見られるように、最後の2フレームだけを soon-to-divide とラベル付けしたにもかかわらず、確率は分裂の約6フレーム前から上昇し始めており、この直線的な増加は実際に起きています。ただし、後の Linker のセクションで見るように、私は2フレームしか持っておらず、時間方向の特徴量を本格的に拡張することはありませんでした。

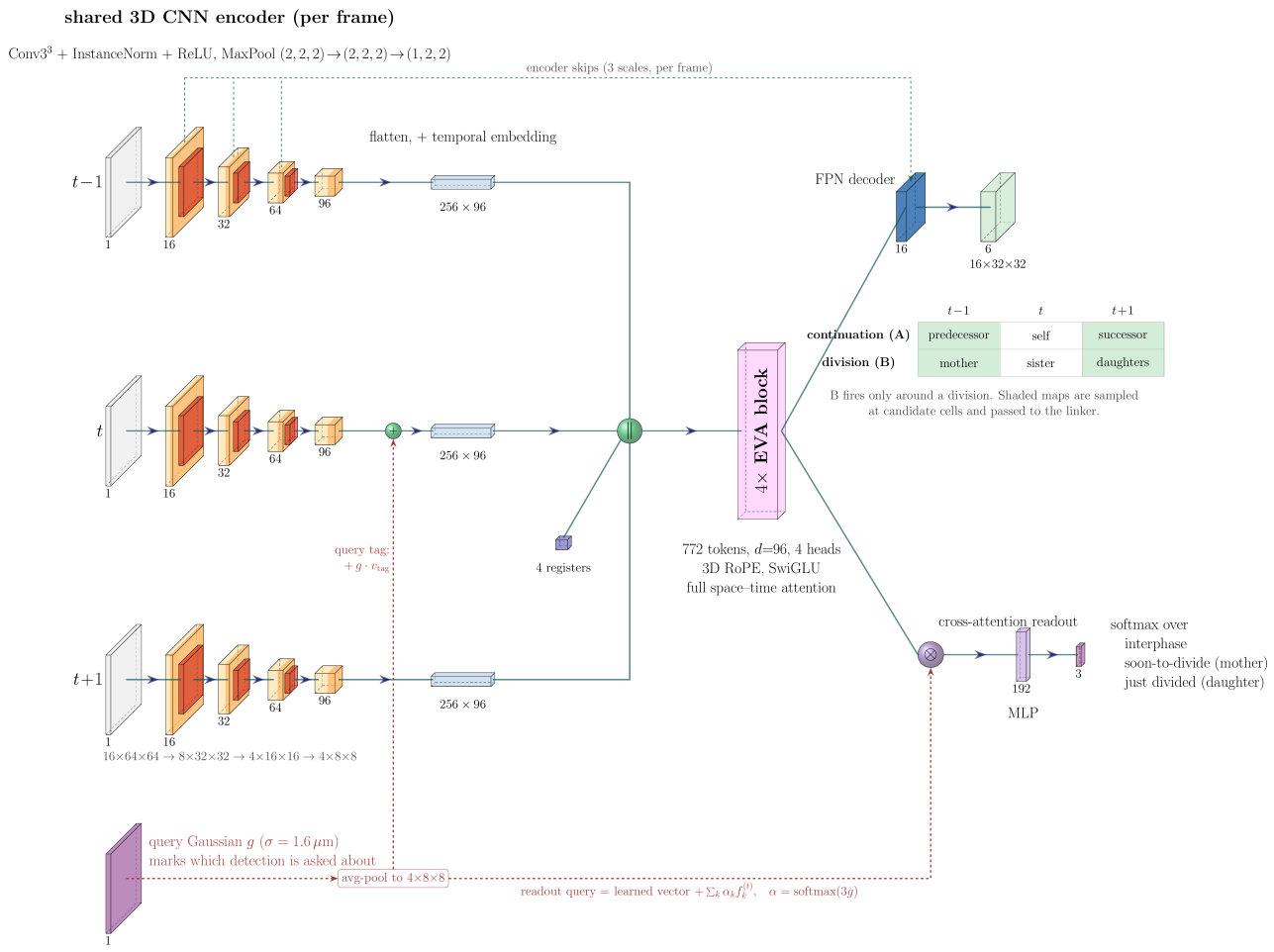


初期のアーキテクチャはシンプルでした。各検出について、フレーム t-1、t、t+1 でその周りの小さな 16×64×64 のクロップを切り出し、フレーム t の細胞位置に Gaussian を置いた4つ目のチャネルを追加して、どの細胞について尋ねているのかをモデルがわかるようにしました。3つのフレームは同じ小さな 3D CNN を通り、小さな transformer が3フレームすべてのトークンに attention をかけ、細胞の位置で初期化された学習可能な query がクラスを読み出します。

soon-to-divide モデルの結果はこの時点で既に良好でした。私は Public LB で ~0.925 付近で行き詰まっていましたが、これにより division の改善だけで Public LB ~0.963 まで上げることができました。しばらくして、データの潜在能力を十分に引き出していないことに気づきました。3フレームを見せ、尋ねたい細胞に query を置いて分類しているだけでしたが、私にはトラックがあったのです！昨年の RSNA コンペでは、@tomoon33 の1位解法 [9] が結果を改善するために補助的な検出ヘッドを使っており、私も似たようなことを試したいと思いました。

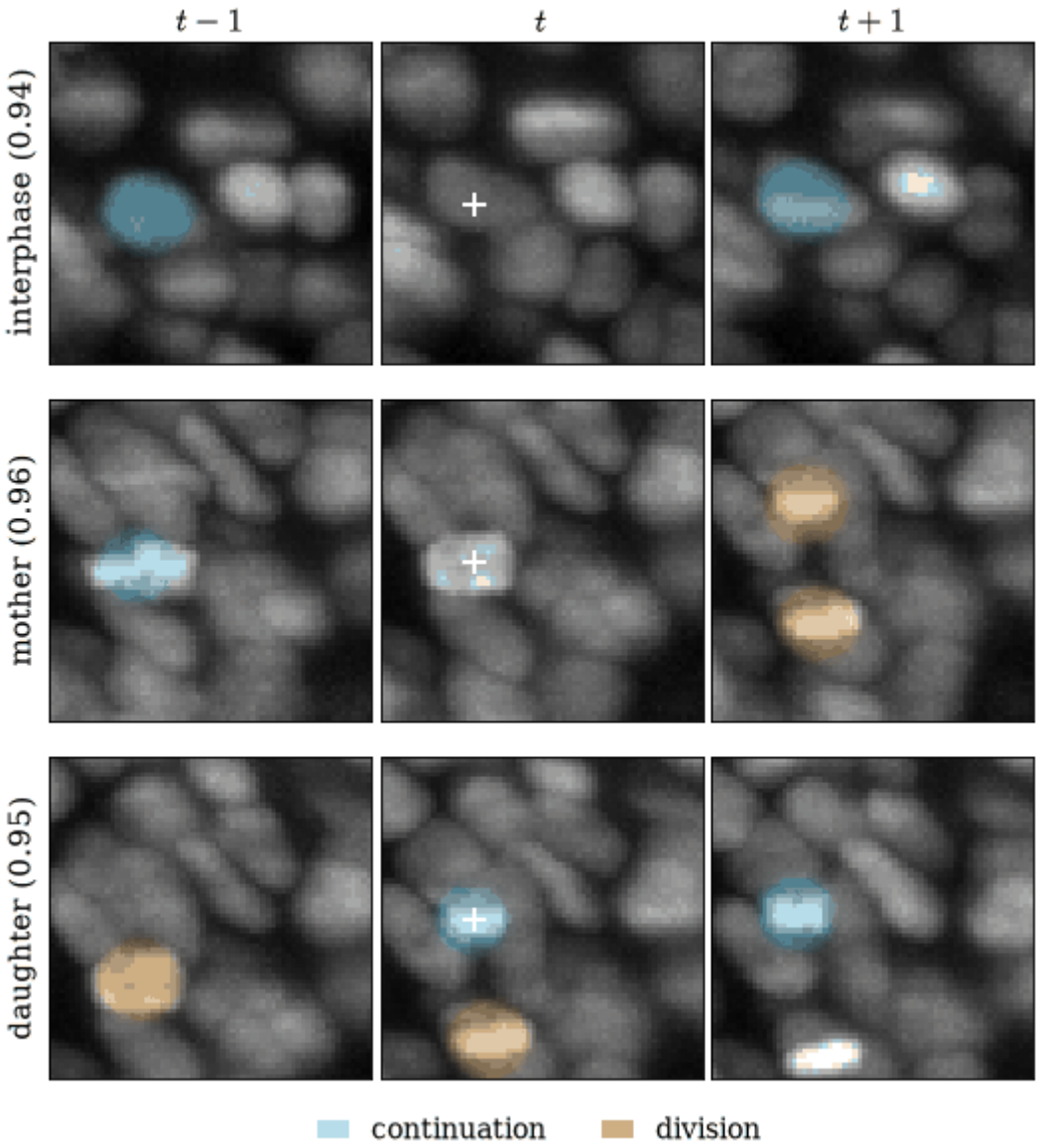
ここで、トラッキングについて私が気になっていた2つ目のことに取り組み始めました。FPN ヘッドを追加することで、モデルは細胞の状態だけでなく、それを見るだけで細胞がどこにいて、どこへ行ったかも言えるようになりました。FPN は各フレームのエンコーダ特徴と transformer のトークンを受け取り、3フレームそれぞれについて、continuation 用と division 用の2つの occupancy map を予測します。ターゲットは、query 細胞とその親族の位置に置いた小さな球です。通常の細胞では、t-1、t、t+1 での位置が continuation map で点灯するので、t+1 の map は「この細胞は次のフレームでここにいるだろう」と言っていることになり、これはまさに linker が答えなければならない問いです。母細胞では t+1 の division map で両方の娘細胞が点灯し、生まれたばかりの娘細胞では t-1 の division map で母細胞が点灯します。

アーキテクチャを下に示します。その多くは MIC-DKFZ（そう、また彼らです）による Primus [10] から来ています。transformer の前にトークナイザとして小さな CNN を使うというアイデアは彼らから初めて学んだものですが、彼らのトークナイザはずっと大きく、簡単に過学習してしまったため、できる限り単純化した結果この小さな CNN に落ち着きました。EVA [11] ブロックも Primus から来ていますが、ここは正直に言わなければなりません。彼らが使っていたから使っただけで、素の transformer ブロックとの適切なアブレーションは行っていません。



occupancy map のアイデアはシンプルですが、うまく機能させるには学習レシピにいくつか調整が必要でした。query 細胞を常にクロップの中心に置くことはできませんでした。そうするとモデルが怠けて、query された細胞ではなく中心にある細胞の occupancy を予測するようになってしまうからです。そのため、非常に大きな FOV ジッター（z 方向 ±3 px、xy 方向 ±16 px）を使っています。また、interphase の細胞に対しては一種のタイムステップ拡張も使っています。フレーム t-1 が実際には t-2 や t-3 であったり、t+1 が t+2 や t+3 であったりすることがあり、これによりモデルは細胞が遠くまで移動し得ることを学習します。

下の例では、ある細胞を query し、その周りで FOV を動かしていますが、ご覧のとおり occupancy map とクラス確率は一貫しています。



Soon Net の初期バージョンを使って division をマイニングし、数百件を目視で確認しました。最終バージョンでは、元の GT の 151 件に対して、515 件の division で学習できました。

クラスヘッドには label smoothing 0.1 を用いた3クラスの cross-entropy を使います。occupancy map には foreground と background を等しく重み付けした BCE に加え、球を含む map に対して Focal-Tversky 項を使います。誤ったタイプの map、たとえば interphase 細胞の division map には、ゴーストブロブを避けるため、値が最も高い 512 ボクセルに追加の BCE をかけます。total loss はクラス loss + 2 × map loss です。

学習は2ラウンドで構成されました。Round 1 は 100 エポック学習し、1エポックあたり約 6650 クロップを、1:1:1（interphase、soon-to-divide、just divided）のバランスで見せました。Round 2 では、Round 1 を使って難しい interphase、すなわち interphase のトラックに属するにもかかわらず Round 1 が soon-to-divide（$p \geq 0.5$）または just divided（$p \geq 0.3$）とスコアリングした細胞を out-of-fold でマイニングし、2492 件の hard negative を得ました。2つ目のバージョンも 100 エポック学習し、1エポックあたり 6600 クロップ、1:1:1:1（ランダムな interphase、難しい interphase、soon-to-divide、just divided）の比率としました。

Soon Net は単体でも、occupancy field 上の greedy デコードを使って既にトラッキングができます。サブミットはしていませんが、5-fold CV でのスコアを以下に示します。元のトラックと、手作業のアノテーション、マイニン

グした division、そして元のトラックで division のアノテーションが欠けていた箇所のいくつかの修正を加えた拡張版トラックの両方での結果です。

GT	公式スコア, Round 1	公式スコア, Round 2	adj. edge Jaccard (R1 / R2)	division Jaccard (R1 / R2)	Divisions TP/FP/FN (R1 → R2)
元の GT	0.9270	**0.9288**	0.8926 / 0.8914	0.3439 / 0.3738	76/73/72 → 77/58/71
拡張 GT	0.9459	**0.9464**	0.8914 / 0.8903	0.5443 / 0.5617	313/70/192 → 314/54/191

まずまずのスコアですが、競争力を持つにはまだ不十分なので、これらの特徴量をより良く使う方法を学習する linker が依然として必要です。なお、Soon Net は最後に登場したピースで、初期の実験は検出 + 学習型 linker だけであり、このモデルはずっと後になって登場しました。

Linker

最後に、学習型 linker です。その役割は、一度に連続する2フレームを見て、どの細胞がどこへ行くのか、そしてどの細胞が分裂したのかを判定することです。主な着想元は LightGlue で、これは2枚の画像間でキーポイントをマッチングするものですが、2フレーム間で細胞をマッチングするのもある意味同じ問題です。

linker は2種類の特徴量を使います。ノード特徴量は単一の細胞を記述し、transformer を通ります。ペア特徴量は2つの細胞間の候補リンクを記述し、後段の MLP ヘッドでのみ使われます。

まずノード特徴量から説明すると、各検出は 42 個の手作り特徴量から構築されるトークンになります。最初の 39 個は線形層を通り、視野（field of view）の端までの3つの距離は専用の小さな線形層を通って同じトークンに加算されます（これが図中の「FoV embedding」です）。はい、手作り特徴量についてあれだけ語っておきながら、linker は依然としてそれを使っています。コンペの序盤には見下していましたが、結果には反論できません、効くんです（笑）。ただし、occupancy 系の特徴量には一つ違いがあります。これらもある意味手作りで、親のヒートマップを子の中心で読み取ると決めているのは私ですが、盲目的ではありません。そのヒートマップは3フレームを見たモデルから来ているからです。したがって、より多くのデータやより大きなモデルで Soon Net が良くなれば、これらの特徴量も良くなります。ここがスケールさせられる部分だと考えています。

次元	特徴量
1	マスクの対数体積
1	マスク上の foreground 確率の平均
24	位置 z, y, x（正弦波、4 周波数 × sin/cos）
13	µm 単位のマスク形状（半軸、軸比、充填率、主軸方向、bounding box サイズ）
3	z, y, x 方向の視野端までの距離

両フレームのトークンは4つの cross-attention ブロックを通ります。各細胞は他方のフレームの近くの細胞、つまり 30 µm 以内のすべての細胞とその最近傍 16 個だけを見るので、疎な領域にある細胞でも常に候補を持つことになります。

次にペア特徴量で、すべての候補リンク（t の親、t+1 の子）について計算されます。そのほとんどは2つの細胞間の単純な差分です。occupancy 系の特徴量は Soon Net の map（Soon Net の図で示したヒートマップ）から来ています。ある候補リンクについて、親に「t+1 ではどこにいると思う？」と尋ねてそのヒートマップを子の中心で読み取り、次に子に「t-1 ではどこにいた？」と尋ねてそのヒートマップを親の中心で読み取ります。両方の答えが高ければ、それはおそらく正しいリンクです。

次元	ペア特徴量
3	変位 z, y, x
1	対数体積比
5	局所的な組織の動きを除去した後のドリフト残差（粗から細へ）
5	形状変化（主軸の一致度、軸長、伸長度）
4	輝度変化
1	親の t+1 ヒートマップを子の位置で読み取った値
1	子の t-1 ヒートマップを親の位置で読み取った値
1	同じ値を他の最良候補と比較したもの
1	子から親のヒートマップのピークまでの距離
2	他方の細胞がクロップ外にある場合のフラグ

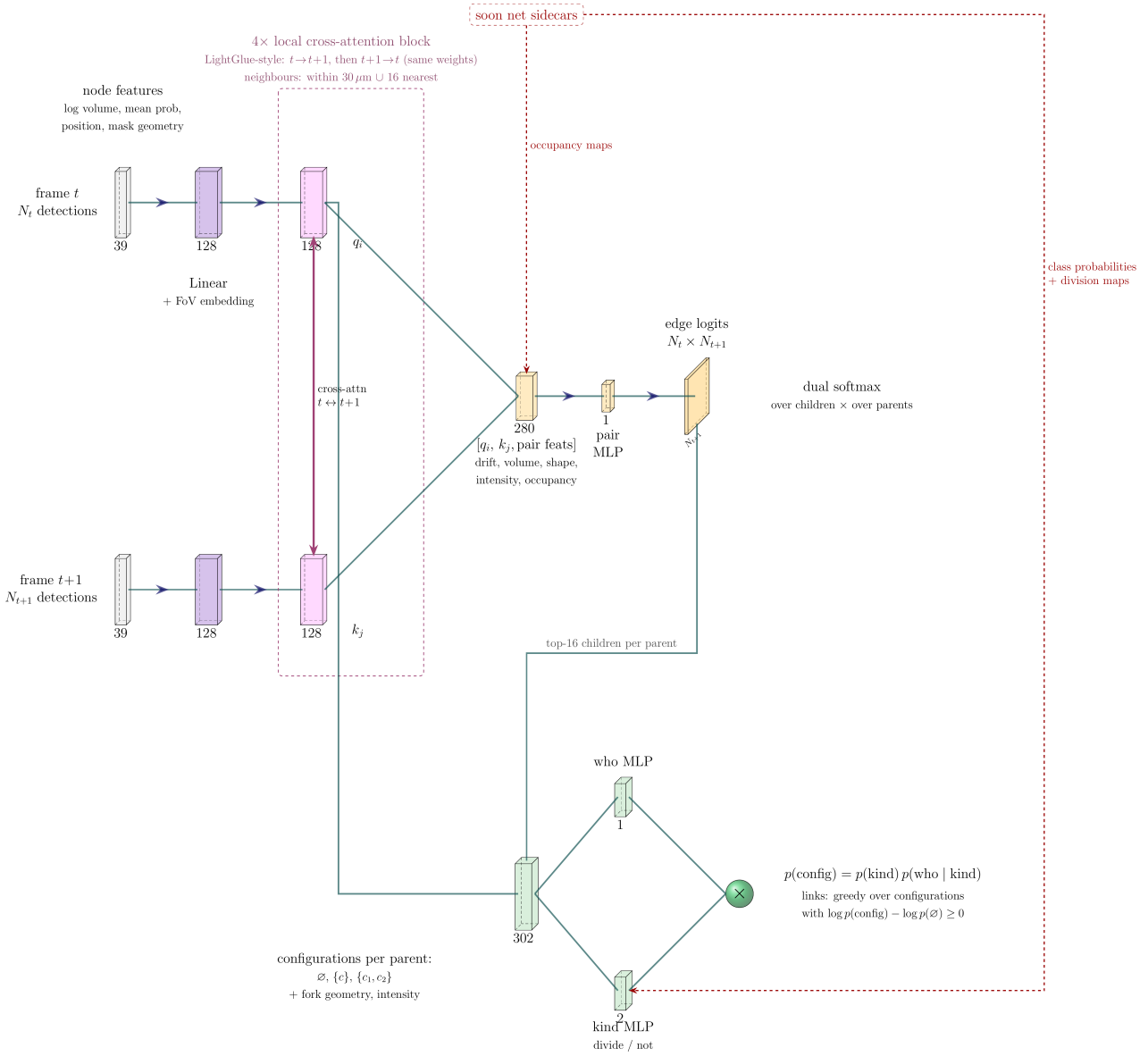
linker には2つのヘッド、pair ヘッドと fork ヘッドがあります。

pair ヘッドは、2つの細胞の埋め込みとペア特徴量を使って、すべての候補リンクをスコアリングします。

fork ヘッドは、各親について最終判断、すなわち子なし、子1つ、子2つのいずれかを決定します。pair スコアで上位 16 個の候補の子を選び、すべての選択肢、つまり子なし（1）、単一の子（16）、子のペア（16×15/2 = 120）をスコアリングするので、親1つあたり 137 個の選択肢になります。

判断は2段階に分けられています。まず「kind」ステップで、親がそもそも分裂するかどうかを判定します。次に「who」ステップで子を選びます。これらは2つの異なる問いなので分割しました。「この細胞は分裂しているか？」は主に細胞そのもの、つまり Soon Net がその細胞について何と言っているかに依存しますが、次のフレームに良い娘細胞のペアがあるかどうかにも依存するので、kind ステップでは最良の候補ペアがどれだけ良いかも見ます。「どの細胞が娘細胞か？」は候補の中から選ぶ問題です。ここで who ステップは、Soon Net の division map（母細胞が両方の娘細胞を指し、両方の娘細胞が母細胞を指し返す）、pair スコア、輝度、そして姉妹細胞間の距離、それらの体積の合計が母細胞と一致するか、分裂が母細胞の主軸に沿って起きているかといったいくつかの形状情報を使います。すべてを単一の softmax にまとめていたときは、120 個のペア選択肢が稀な division の判断をかき消してしまい、モデルは誤った division を多く生成しすぎていました。分割することで division の判断が専用の小さなヘッドを持つようになり、最終的な確率は単純に $p(\text{kind}) \times p(\text{who} \mid \text{kind})$ となります。

起きていることすべてを追うのはやや複雑なので、下の図でより分かりやすく示しています。



linker には等しい重みの3つの loss があります。pair ヘッドは LightGlue と同様に dual-softmax NLL を使います。fork ヘッドは各親の 137 個の選択肢に対する NLL を使い、kind と who のステップを一緒に学習します。3つ目の loss は逆方向で、これも NLL であり、各子はそれを主張するすべての選択肢の中から正しい親、または親なしを選ばなければなりません。

学習は拡張 GT 上で 10 エポック行い、division を 8 倍にオーバーサンプリングしました。タイムステップ拡張も使っており、25% の確率で (t, t+2) のペアを見ることになります。

推論時には 5 fold の linker を平均し、リンクは greedy デコードから得ます。edge スコアに閾値はかけません。すべての親とすべての選択肢について、デコードは「子なし」よりどれだけ良いか、log p(option) − log p(no child) を計算し、0 より大きいものだけを残します。これらをソートして最良のものから順に採用し、各親は最大1つの選択肢を取り、各子は1つの親しか持てません。親が見つからなかった子は新しいトラックを開始します。

後処理

後処理はできる限りシンプルに保つよう努めました。公開ノートブックで共有されていた line-fit スムージング（division を母細胞へとスムージングしないよう調整したもの）、生まれたばかりの娘細胞が次のフレームで再び

分裂することを許さない division 不応期ルール（競合は最も確信度の高い division から順に解決します。そうしないと、弱い早期の fork が1フレーム後の本物の division をブロックしてしまう可能性があるためです）、1フレームのギャップ補完、そして 3 フレームより短いトラックの刈り込みです。

パイプライン全体（検出器 + Soon Net + linker、greedy デコード）の最終的な 5-fold CV を以下に示します。

GT	Soon Net 単体	パイプライン全体	adj. edge Jaccard (単体 / 全体)	division Jaccard (単体 / 全体)	Divisions TP/FP/FN (全体)
元の GT	0.9288	**0.9575**	0.8914 / 0.9089	0.3738 / 0.4857	102/62/46
拡張 GT	0.9464	**0.9856**	0.8903 / 0.9090	0.5617 / 0.7654	434/62/71

最終スコアは Public LB で 0.976、Private LB で 0.977 です。

コード

公開予定

参考文献

[1] Jordão Bragantini, ultrack: large-scale versatile cell tracking in Python, SciPy 2024. https://www.youtube.com/watch?v=98dahngkNOI

[2] Stringer et al., Cellpose: a generalist algorithm for cellular segmentation, Nature Methods (2021). https://github.com/MouseLand/cellpose

[3] He et al., Masked Autoencoders Are Scalable Vision Learners, CVPR (2022). https://arxiv.org/abs/2111.06377

[4] Lindenberger, Sarlin, Pollefeys, LightGlue: Local Feature Matching at Light Speed, ICCV (2023). https://arxiv.org/abs/2306.13643

[5] Bragantini, Theodoro, Royer, Higher-Order Cell Tracking Transformer, arXiv:2607.11754 (2026). https://arxiv.org/abs/2607.11754

[6] Bragantini, Theodoro, Zhao et al., Ultrack: pushing the limits of cell tracking across biological scales, Nature Methods 22, 2423–2436 (2025). https://doi.org/10.1038/s41592-025-02778-0

[7] Zhang, Mu, Liu et al., FOCUS-3D: Robust, generalizable volumetric cell segmentation for three-dimensional fluorescence microscopy, bioRxiv (2026). コンペのディスカッションで @hengck23 が共有。

[8] MIC-DKFZ, nnssl, 自己教師あり学習フレームワーク（ここで使用した MAE のコード）. https://github.com/MIC-DKFZ/nnssl

[9] tomoon33, RSNA Intracranial Aneurysm Detection, 1st place solution, Kaggle (2025). https://www.kaggle.com/competitions/rsna-intracranial-aneurysm-detection/writeups/1st-place-solution

[10] Wald, Roy, Isensee et al., Primus: Enforcing Attention Usage for 3D Medical Image Segmentation (2025). https://arxiv.org/abs/2503.01835

[11] Fang et al., EVA-02: A Visual Representation for Neon Genesis (2023). https://arxiv.org/abs/2303.11331